



020 – MASTER AUDIT + VERIFY + FIX + COMPLETE + PRODUCTION PACKAGE PROMPT

(Node/Express/TypeScript + Drizzle/PostgreSQL + React Native CLI monorepo – Android + Web only)

Paste this entire file into Replit's Agent/Assistant chat with the project workspace open.

This is **not** a "build from scratch" prompt. This is an **audit** → **fix** → **complete** → **package** prompt for a project that is already *partially* built on the stack described below. Treat nothing as finished until you personally verify it by reading the actual code and exercising the actual feature.

STACK LOCK – READ THIS FIRST: This project uses a specific existing stack (Node/Express/TypeScript, Drizzle ORM, PostgreSQL, React Native CLI, Socket.IO, pnpm workspaces). **Do not introduce Django, Python, AWS/S3, Redis, Celery, Expo, or any other framework/service not already in this stack.** Do not add any new major dependency (a new ORM, a new realtime library, a new cloud storage SDK, a new charting library, etc.) without first checking whether an equivalent already exists in the monorepo's `package.json` files. If something is genuinely missing and required, implement it using the lightest-weight option compatible with the existing stack (see specific guidance per feature below) – never by pulling in an external managed service.



0. PROJECT / WORKSPACE TO ATTACH

Open/attach the full **020 pnpm monorepo** to the Replit workspace. This is the **actual, confirmed** top-level layout (from a real directory listing) – treat this as ground truth over Section 1's earlier generic description wherever the two differ:

```
020/
├── .agents/                ← agent/tooling config (inspect before
touching – don't delete blindly)
├── .git/
├── .local/                 ← local-only files (inspect contents;
typically not part of the shipped app)
└── artifacts/
```

```

|   └─ admin-panel/                ← React 19 + Vite + Wouter + Recharts
admin dashboard
|   └─ api-server/                 ← Node.js + Express (v5) + TypeScript
backend
|   └─ mockup-sandbox/             ← design/mockup sandbox – inspect its
purpose; likely not part of the
|   |                               production build, but do not delete
without confirming it's unused
|   └─ o2o/                        ← React Native CLI app (Android + Web via
React Native Web/Vite)
└─ attached_assets/
|   └─ *.pdf                       ← original spec/reference document – read
this, it may be the
|   |                               authoritative source of truth for
feature scope
|   └─ Pasted-PROJECT-NAME-O2O-Marketplace-Platform-Tech-Stack-
Frontend_*.txt
|                                   ← pasted original tech-stack/spec text
– READ THIS FIRST, it likely
|                                   defines the canonical stack and
feature list for this project
└─ b/
|   └─ Debug/
|   └─ tools/                      ← inspect purpose before treating as dead
code; do not delete blindly
└─ lib/
|   └─ api-client-react/           ← shared React Query hooks / API client
used by mobile + admin
|   └─ api-spec/                  ← shared API contract/spec package
(verify how it's consumed – likely
|   |                               feeds validation/types for both api-
server and the frontends)
|   └─ api-zod/                   ← shared Zod schemas (separate from
drizzle-zod – verify how this
|   |                               relates to lib/db and lib/api-spec;
don't create a duplicate)
|   └─ db/                        ← Drizzle ORM schema, migrations,
drizzle-kit config
└─ node_modules/
└─ scripts/
|   └─ src/

```

```

|   └─ package.json
|   └─ post-merge.sh           ← git post-merge hook script – leave in
place unless proven dead
|   └─ tsconfig.json
└─ .env
└─ .eslintrc.json
└─ eslint.config.mjs
└─ package.json               ← root workspace config
└─ pnpm-lock.yaml
└─ pnpm-workspace.yaml
└─ README.md
└─ test-e2e.mjs               ← root-level E2E test script – check what
it covers before assuming
|                               Phase 4 testing needs to start from
scratch
└─ test-extended.mjs          ← root-level extended test script – same
as above
└─ tsconfig.base.json
└─ tsconfig.json

```

Before doing anything else: open and read `attached_assets/Pasted-PROJECT-NAME-O2O-Marketplace-Platform-Tech-Stack-Fronte_*.txt` and the PDF in `attached_assets/`. These are very likely the original project brief – cross-check everything in this prompt against them and flag any conflicts in `AUDIT_REPORT.md` rather than silently picking one source over the other.

(Optional but helpful) – any previous audit report, screenshots of currently broken screens, or `.env.example` files with real (redacted) values so Replit knows what's already configured.

Do not attach unrelated design mockups or unrelated repos beyond what's already in `attached_assets/` and `artifacts/mockup-sandbox/`. This monorepo is the only source of truth. Folders whose purpose isn't obvious (`.agents/` , `.local/` , `b/` , `artifacts/mockup-sandbox/`) must be **inspected and understood before any deletion or rewrite** – do not assume they're dead code just because they weren't mentioned in earlier planning docs.



1. CONFIRMED TECH STACK – DO NOT DEVIATE

Monorepo / Tooling

Package manager: pnpm (pnpm workspaces)

Language: TypeScript, strictly typed, across the entire monorepo

Database Layer (lib/db)

Database engine: PostgreSQL (via the `pg` driver)

ORM / Query builder: Drizzle ORM (drizzle-orm) + drizzle-kit for schema/migrations

Validation: Zod, via drizzle-zod generating schemas from DB models

Backend API Server (artifacts/api-server)

Runtime: Node.js

Framework: Express.js v5

Language: TypeScript

Logging: Pino + Pino-HTTP (structured logging)

Security/middleware: Helmet, CORS, express-rate-limit

Authentication: JSON Web Tokens (jsonwebtoken) – access + refresh token pattern

Password hashing: bcrypt (or equivalent already present in the repo – verify, don't add a new one)

File uploads: Multer → stored on LOCAL DISK on the API server, served via an Express static route

(NOT AWS S3 – no cloud storage SDK is part of this stack)

Realtime: Socket.IO (server) attached to the same Express HTTP server

(NOT Django Channels, NOT raw `ws`, NOT any managed pub/sub service)

Scheduled/background jobs: node-cron (or an equivalent lightweight in-process scheduler

already present in the repo) – NOT Celery, NOT BullMQ/Redis-backed queues

Build tool: ESBUILD

Frontend – Mobile + Web App (artifacts/o2o)

Framework: React Native CLI (bare workflow – NOT Expo), targeting Android + Web only

(Web target rendered via React Native Web, bundled with Vite)

UI: React 19

Navigation: React Navigation v7 (Native Stack + Bottom Tabs)

State & data fetching: React Query (@tanstack/react-query) via the shared

lib/api-client-react package

Realtime client: socket.io-client

Animations: React Native Reanimated

Storage: @react-native-async-storage/async-storage (Android) / browser localStorage (Web)

– NOT expo-secure-store (not part of a bare RN CLI project)

Image picking / camera: whatever bare-RN-compatible library is already in the repo
(e.g. react-native-image-picker) – NOT expo-image-picker
Push notifications (Android only): whatever Firebase Android integration is already wired in the repo (e.g. @react-native-firebase/messaging) – NOT an Expo push service

Frontend – Admin Panel (artifacts/admin-panel)

Framework: React 19
Bundler/build: Vite
Routing: Wouter
State & data fetching: React Query (@tanstack/react-query)
Data visualization: Recharts

Explicitly OUT of scope / must never be (re)introduced:

- ✗ Django, Python, any Python-based backend code
 - ✗ AWS / S3 / any cloud object storage SDK
 - ✗ Redis, Celery, Celery Beat, or any Redis-backed queue
 - ✗ Expo, Expo Go, `app.json` Expo config, any `expo-\*` package
 - ✗ iOS support of any kind (see Section 4)
-

2. ABSOLUTE GROUND RULES – READ FIRST, FOLLOW ALWAYS

1. ASSUME NOTHING IS FINISHED. Every screen, every endpoint, every DB schema file
must be personally opened and read before you decide it "already works."
2. NEVER report something as done because a file with the right name exists.
A file existing is not the same as a feature working. Trace the actual logic.
3. DO NOT skip verification. Read every file before deciding it works.
4. ZERO mock/hardcoded data, ZERO placeholder values, ZERO "TODO" comments,
ZERO empty onPress/onClick handlers, ZERO fake chart data, ZERO stub functions
anywhere in the final app. If a screen shows mock/hardcoded data, it is broken –

- fix it to use the real API.
5. If a feature is partially implemented, it is NOT implemented. Fix it completely.
 6. THIS APP TARGETS ANDROID AND WEB ONLY. iOS does NOT exist in this project.

Remove every line of iOS-specific code – see Section 4. Do not write one line of new iOS-specific code, do not keep iOS native project folders or scripts, do not test against iOS, do not run `react-native run-ios`.
 7. STAY ON THE CONFIRMED STACK (Section 1). Do not add Django/Python/AWS/Redis/Celery/Expo. Do not add a new ORM, new realtime library, new cloud storage SDK, or new major dependency without first confirming nothing equivalent already exists in the relevant `package.json`.
 8. DO NOT rewrite screens/modules that are already correctly implemented. Read first, confirm correctness, leave alone. Only touch what is broken, missing, incomplete, or mocked.
 9. DO NOT rename existing files, DB tables/columns, endpoints, Socket.IO event names, or navigation routes unless a rename is strictly required to fix a real bug (e.g. duplicate file collision) – and if you do, fix every import/reference across the monorepo.
 10. DO NOT change API URL paths that already work end-to-end.
 11. Never trust the frontend for validation or business rules – every rule (role checks, bid timing, review eligibility, offer editability, etc.) must be enforced server-side (Express route handlers / middleware), not just hidden in the UI. Every backend endpoint must validate permissions server-side.
 12. Every API call from the frontend must include a JWT Bearer token. Every Socket.IO connection must authenticate via the handshake `auth` payload (not a bare query string, unless that's already the established

pattern

in the codebase – verify first).

13. Every fix must be verified with an actual run/test before you move to the next item – do not batch five fixes and hope they all work.

14. The final deliverable is ONE downloadable, production-ready archive (or a clean Replit-runnable workspace) that:

- installs cleanly with `pnpm install` at the repo root
- the API server starts with 0 TypeScript errors and 0 runtime crashes
- the Drizzle schema/migrations apply cleanly to a fresh PostgreSQL DB
- the mobile app builds/runs on Android via React Native CLI with 0

crashes

- the web target builds via Vite with 0 errors
- the admin panel builds via Vite with 0 errors
- contains no dead code, no unused duplicate files, no iOS artifacts, no Django/Python/AWS/Redis/Expo artifacts

3. PROJECT OVERVIEW

App Name: 020 (Buy • Sell • Bid)

Type: Social-commerce + reverse marketplace

Monorepo manager: pnpm workspaces, fully TypeScript

Mobile + Web frontend: React Native CLI app ([artifacts/o2o](#)) – Android native + React Native Web (bundled by Vite) – **iOS removed**

Admin panel: React 19 + Vite + Wouter + Recharts ([artifacts/admin-panel](#))

Backend: Node.js + Express v5 + TypeScript ([artifacts/api-server](#))

Database: PostgreSQL, accessed via Drizzle ORM ([lib/db](#)), validated with Zod/drizzle-zod

Realtime: Socket.IO (server in [api-server](#) , client via [socket.io-client](#) in [o2o](#))

File storage: Local disk on the API server, served via an Express static route

Auth: JWT (access + refresh) via [jsonwebtoken](#)

Scheduled jobs: node-cron (or existing lightweight in-process scheduler) – used for bid auto-close, OTP expiry cleanup, etc.

Push: Firebase Cloud Messaging, Android only, via whatever bare-RN Firebase integration already exists in the repo

Note: this section describes the confirmed target stack (Section 1). Verify the actual code against it – flag in [AUDIT_REPORT.md](#) anywhere the codebase drifts from this stack (e.g. a leftover Expo config, a stray Python file, an AWS SDK import) so it can be removed.

4. PHASE 0 – INTAKE & INSPECTION (do this before touching any code)

Before writing a single line of code:

1. List the monorepo folder structure at least 2 levels deep for `lib/db`, `lib/api-client-react`, `artifacts/api-server`, `artifacts/o2o`, and `artifacts/admin-panel`.
2. Confirm the actual stack matches Section 1. Report any drift (leftover Expo config, Python files, AWS SDK imports, Redis client imports, etc.) – these must be flagged for removal, not treated as "the real stack."
3. In `artifacts/o2o`, read every file in `src/screens/` (or equivalent) and every file in `src/api/` or the shared `lib/api-client-react` package. For each screen, note:
 - Does it call a real API (via React Query + the shared API client), or does it use mock/hardcoded data?
 - Are all `onPress` / `onClick` handlers wired to real logic, or empty/ `console.log` / `TODO`?
 - Does it reference any undefined variable, function, or mutation (grep for likely typos/undefined refs)?
 - Does it import anything Expo-specific or iOS-specific?
4. In `artifacts/api-server`, read every route/controller file, every Drizzle schema file in `lib/db`, every Socket.IO event handler, and every scheduled job. Build a full endpoint inventory (method + path + what it actually does, not what a comment claims it does) and a full Socket.IO event inventory (event name + namespace/room + what it actually does).
5. Cross-reference: for every frontend API call / Socket.IO event listener, confirm the matching backend route/event exists and actually implements the described behavior (open the handler code, don't just check the router file).
6. Produce an **Audit Report** (write this to `AUDIT_REPORT.md` in the repo root) listing, for every screen/feature in the checklists in this document:
 -  Implemented & verified correct
 -  Implemented but broken/incomplete/mocked (describe exactly what's wrong)
 -  Not implemented at all
 -  Off-stack (uses Django/Python/AWS/Redis/Expo or another disallowed dependency – must be migrated to the confirmed stack)
7. Only after this report exists, proceed to iOS Removal, then Phase 1 (fixes), working strictly in priority order:  Critical →  High →  Medium →  Low.

Do not skip Phase 0. If you jump straight to "fixing," you will re-break working code or waste effort re-implementing things that already work.



5. iOS REMOVAL – MANDATORY, DO THIS EARLY (Before Any Feature Work)

The app must run **only on Android and Web**. Since this is a **React Native CLI (bare)** project, there is a real native `ios/` Xcode project to remove – this is not an Expo `app.json` config toggle.

Delete these folders/files if they exist:

<code>artifacts/o2o/ios/</code>	← delete the entire native iOS
<code>Xcode project folder</code>	
<code>artifacts/o2o/**/*.ios.js</code>	← delete all iOS-specific files
<code>artifacts/o2o/**/*.ios.jsx</code>	← delete all iOS-specific files
<code>artifacts/o2o/**/*.ios.ts(x)</code>	← delete all iOS-specific files
<code>Podfile / Podfile.lock (iOS CocoaPods)</code>	← delete, since they only apply to ios/

In `artifacts/o2o/package.json`:

- Remove any `"ios"` script (e.g. `"ios": "react-native run-ios"`)
- Remove any iOS-only npm packages present purely for iOS (iOS-only permission libs, APNs-specific push setup). Keep cross-platform packages that also work on Android/Web.

In `metro.config.js` / `vite.config.ts` / any bundler config:

- Remove any iOS platform extensions/aliases (`.ios.js` resolution, iOS-only platform lists) – keep only `android` and `web` (and default/shared) platform resolution.

In every `.tsx` / `.ts` / `.jsx` / `.js` file – search and replace:

REMOVE: `Platform.OS === 'ios'` checks – collapse logic to only handle `'android'` and `'web'` (and a shared default). Do not leave dead iOS branches "just in case."

REMOVE: Any iOS-only storage/keychain library (this stack should already be using `@react-native-async-storage/async-storage` on Android and `localStorage` on Web – see `TokenStorage` utility below).

REMOVE: `KeyboardAvoidingView behavior="padding"` (iOS-only pattern)
Replace with: `behavior="height"` for Android, `no-op/skip` for Web

REMOVE: Any import that is iOS-only (`ActionSheetIOS`, iOS-specific native

modules)

KEEP: react-native-safe-area-context (works on Android too)

Token Storage utility (Android + Web only, no iOS Keychain):

```
import AsyncStorage from '@react-native-async-storage/async-storage'
import { Platform } from 'react-native'

const TokenStorage = {
  async set(key: string, value: string) {
    if (Platform.OS === 'web') {
      localStorage.setItem(key, value)
    } else {
      await AsyncStorage.setItem(key, value)
    }
  },
  async get(key: string) {
    if (Platform.OS === 'web') {
      return localStorage.getItem(key)
    }
    return await AsyncStorage.getItem(key)
  },
  async remove(key: string) {
    if (Platform.OS === 'web') {
      localStorage.removeItem(key)
    } else {
      await AsyncStorage.removeItem(key)
    }
  }
}
```

Documentation cleanup:

- Update any README, setup scripts, or docs that mention iOS/Xcode/CocoaPods/Simulator — delete those sections.

After iOS removal, verify (must all return zero/false results):

```
# No native iOS project remains
test -d artifacts/o2o/ios && echo "FAIL: ios/ still exists" || echo "OK:"

# No iOS platform checks left in code
grep -rn "Platform.OS === 'ios'" artifacts/o2o/src --include="*.ts" --inc
```

```
# No iOS script left in package.json
```

```
grep -n "\"ios\"" artifacts/o2o/package.json
```

```
# No stray iOS references (ignore incidental substring matches like "vers
```

```
grep -ri "ios" artifacts/o2o/src
```

- Confirm `pnpm --filter o2o run android` (or the repo's actual Android run script) and the web dev/build script (Vite) both work after removal. Do **not** attempt any `run-ios` command at all – it is out of scope and should not be referenced anywhere, including in `package.json` scripts, CI config, or documentation.

All checks must pass before proceeding to feature work.

6. PHASE 1 – CRITICAL FIXES (fix these first, in this order)

For each item: locate the real file, read it fully, verify against the rule, then implement completely (not partially) if broken/missing, using the confirmed stack only. Verify with a real test after each fix.

1. **Duplicate/placeholder screen files** – if the same screen name exists in two locations (e.g. a real implementation and a leftover placeholder), delete the placeholder, fix every broken import across the codebase, confirm navigation still resolves to the real one. (Notably: ensure only ONE `OrderChatScreen` file exists, at `src/screens/orders/OrderChatScreen.tsx` – no duplicate at `src/screens/chat/OrderChatScreen.tsx`.)
2. **Undefined mutations/handlers referenced but never defined** – grep the whole `artifacts/o2o` app for any variable/function used in JSX(`onPress={...}`) that has no matching declaration in that file or its imports. Define it properly using the same React Query pattern(`useMutation` from `@tanstack/react-query`, via the shared `lib/api-client-react` client) already used elsewhere in the codebase. Wire it to the real backend Express endpoint (verify the endpoint exists and works – build it server-side first if it's missing, including proper permission checks).
 - **Specifically verify `BuyerOfferSelectionScreen.tsx`**: `selectWinnerMutation` must be defined using `useMutation`, must NOT be undefined or reference an undefined variable. The SELECT button must call `selectWinnerMutation.mutate({ bidId, offerId })`, show a loading spinner on the tapped button while disabling others, and on success navigate to `OrderRequestScreen` with the offer data. If `selectWinnerMutation` is undefined, define it completely.

3. **Wishlist add/remove** – confirm add and remove both call real POST/DELETE Express endpoints (`DELETE /api/v1/wishlist/:productId` must exist and work against Drizzle/Postgres), update UI (optimistic removal – item disappears immediately, restores on error), and that the backend decrements/increments `wishlist_count` on the product row correctly (via a DB trigger, a transaction, or an explicit update in the same handler – verify whichever pattern is already used elsewhere in the schema). Include loading skeleton and empty state. A "not implemented" screen with no API call must be fixed.
 4. **Review submission flow (`ReviewScreen.tsx`)** – verify it is not a placeholder. Star rating, text, and photo upload must all work end-to-end:
 - Interactive 5-star rating, works Android + Web
 - Review text: minimum 10 characters, character count display
 - Photo upload (up to 3 photos): the existing bare-RN image picker library → `POST /api/v1/uploads` (Multer, writing to local disk) → collect the real served URLs (e.g. `/uploads/reviews/<file>`) before submitting the review
 - Submit calls real `POST /api/v1/reviews` , which enforces server-side:
`order.status === 'completed'` (else 400 `ORDER_NOT_COMPLETE`),
`order.buyerId === request.user.id` (else 403), one review per order (unique DB constraint, no duplicates), rating integer 1–5, and creates a notification row for the seller on success
 - Loading overlay during submission; success → navigate back + success alert; error → error alert
 5. **Any screen showing "coming soon" that the spec requires to be fully functional** – implement it for real. Only keep "coming soon" for features explicitly marked optional/future (e.g. two-factor auth toggle) – everything else must work.
-

7. PHASE 2 – HIGH PRIORITY FIXES

7.1 Authentication

Must be JWT-based (access + refresh), issued by the Express API server via `jsonwebtoken` , backed by Drizzle/Postgres user records.

Backend – check `artifacts/api-server/src/routes/auth.ts` (or equivalent) and its controllers:

`POST /api/v1/auth/register`

Validate username/email/mobile unique (DB-level unique constraints + explicit checks),

password strength, role = buyer|seller only (never self-assignable to

admin)

Hash password with bcrypt before storing

Return { accessToken, refreshToken, user: { id, username, fullName, email, role, city } }

POST /api/v1/auth/login

Return JWT access + refresh tokens + user object

POST /api/v1/auth/logout

Accept { refreshToken }, invalidate/blacklist it (DB-backed revoked-token table

or a `revokedAt` column – whichever pattern already exists), return 200

POST /api/v1/auth/token/refresh

Accept { refreshToken }, return new { accessToken, refreshToken }, invalidate old

POST /api/v1/auth/forgot-password

Send 6-digit OTP to email, expire in 10 min (DB row with expiry timestamp),

rate limit 3/hour/email (express-rate-limit or a DB-backed counter)

POST /api/v1/auth/verify-otp

Validate code+email, mark OTP row as used, reject expired/used OTPs

POST /api/v1/auth/reset-password

Set new password (bcrypt-hashed) after OTP verified

POST /api/v1/auth/change-password

Authenticated, validate current password, set new password

GET /api/v1/auth/me

Return full user profile with role

PATCH /api/v1/auth/me

Update fullName, city, profileImage

Frontend([artifacts/o2o](#)):

SplashScreen.tsx

Check token via TokenStorage utility (Android: AsyncStorage / Web:

localStorage)

If valid → Home; if 401 → refresh → Home; if no token → Welcome
NO Platform.OS === 'ios' code

LoginScreen.tsx

Username + password fields with show/hide toggle
Calls POST /api/v1/auth/login via the shared lib/api-client-react client
Stores tokens via TokenStorage utility
Navigates to Home on success, shows field errors on fail

SignupScreen.tsx

All required fields + role selection (Buyer/Seller cards)
Button disabled until all valid + role selected
Calls POST /api/v1/auth/register

ForgotPasswordStep1Screen.tsx / Step2 / Step3

3-step flow: email → OTP verify → new password
OTP input works on Android and Web
Countdown timer visible

Also verify:

- Every authenticated API call attaches `Authorization: Bearer <accessToken>` (via the shared API client's interceptor)
- 401 triggers an automatic refresh-and-retry, and a proper redirect to login if refresh also fails
- Logout invalidates the refresh token server-side
- Socket.IO connections authenticate via the handshake `auth: { token }` payload, validated server-side in Socket.IO middleware before allowing the connection

7.2 Home Screen

HomeScreen.tsx

Real data from the API (not mock)
Recent Chats, Groups, Channels sections with skeleton loading
Seller-only: "Your Channels" + Create Channel button
Buyer-only: "My Bids" shortcut
Bottom navigation (React Navigation Bottom Tabs) visible and functional
Search bar tappable → SearchScreen

7.3 Channels

Backend – check `artifacts/api-server/src/routes/channels.ts` :

```
GET/POST /api/v1/channels
GET/PATCH/DELETE /api/v1/channels/:id
POST/DELETE /api/v1/channels/:id/follow
GET /api/v1/channels/my
    POST restricted to seller role only (auth middleware)
    PATCH/DELETE restricted to owner only
```

Frontend:

ChannelListScreen.tsx – real data, search, skeleton, "+ Create" for sellers only
CreateChannelScreen.tsx – seller only, logo upload via Multer → local disk, all fields required
ChannelViewScreen.tsx – buyer: read-only feed; seller/owner: post composer at bottom

7.4 Products

Backend – check `artifacts/api-server/src/routes/products.ts` :

```
GET/POST /api/v1/products
GET/PATCH/DELETE /api/v1/products/:id
POST /api/v1/products/:id/repost
POST /api/v1/products/:id/view
GET /api/v1/products/channel/:channelId
POST /api/v1/uploads ← Multer, writes to local disk under a served
static
    directory (e.g. /uploads), returns { url }
    POST /products restricted to seller only
    repost copies all fields, images, details → returns a full new product
record
    (real DB insert, not a shallow reference)
```

Frontend:

CreateProductPostScreen.tsx
 Seller only, multi-image upload via POST /api/v1/uploads, custom key/value details

ProductDetailScreen.tsx
 Image carousel, wishlist heart, real API data, real view counter

increment

Buyer sees "Add to Wishlist" + "Create Bid" (not for own products)

Seller sees "Repost" (own products only)

RepostProductScreen.tsx ← COMMON FAILURE POINT

Calls POST /api/v1/products/:id/repost on mount (real server-side DB duplication)

Form pre-fills from REAL API response (zero mock data)

All fields editable, REPUBLISH POST → PATCH /api/v1/products/:newId (persists changes for real)

Loading state while fetching, error state with retry

If still showing mock data → fix it completely

7.5 Seller Analytics

Backend:

Open artifacts/api-server/src/routes/analytics.ts (or equivalent) and verify it

returns REAL computed data via Drizzle queries, matching a manual DB query:

activeBids, totalBids, wonBids, winRate
averageOfferPrice, channelRating, totalReviews
totalFollowers, totalProducts, completedOrders

If any field returns 0 or null incorrectly → fix the Drizzle query (check joins/aggregations, not just the route handler)

Frontend:

SellerAnalyticsScreen.tsx

Check for hardcoded/mock numbers or static chart arrays → replace with real API data

Verify React Query hooks fetching all relevant endpoints via lib/api-client-react:

```
useQuery(['seller-analytics'], analyticsApi.getSellerAnalytics)  
useQuery(['seller-bid-analytics'], analyticsApi.getSellerBidAnalytics)  
useQuery(['seller-review-analytics'],  
analyticsApi.getSellerReviewAnalytics)
```

Map each stat card to the correct API field

For charts: check the existing package.json in artifacts/o2o first – if a

charting library is already installed, use it. If none exists, do NOT add a new heavy charting dependency; implement simple numeric stat cards plus a lightweight custom bar/line visualization using whatever SVG/graphics primitives are already available in the app (e.g. react-native-svg, if present), or flag the gap in AUDIT_REPORT.md for a product decision. Include loading, error, and pull-to-refresh states

7.6 Settings Screen

Backend – check if these endpoints exist, create if missing (Drizzle schema + Express routes):

```
GET/PATCH /api/v1/accounts/notification-preferences
GET/PATCH /api/v1/accounts/privacy-settings
GET /api/v1/accounts/sessions
DELETE /api/v1/accounts/sessions/:id
POST /api/v1/accounts/delete-account
```

If missing:

- notificationPreferences table (1:1 with users, Drizzle schema in lib/db)
- privacySettings table (1:1 with users, Drizzle schema in lib/db)
- Auto-create default rows on user creation (in the same transaction as registration, or via a DB trigger if that pattern already exists)
- Add routes, Zod validation (via drizzle-zod), and controllers; run drizzle-kit migrations

Frontend:

SettingsScreen.tsx

Every onPress handler must do something real – no empty () => {}, no console.log, no null:

- Account: Edit Profile nav, Change Password nav
- Notifications: real useQuery + optimistic useMutation toggles, persisted server-side
- Privacy: real useQuery + optimistic useMutation toggles, persisted server-side
- Appearance: Light/Dark/System, stored via the TokenStorage-style local persistence (AsyncStorage on Android / localStorage on Web)
- Security: Login Sessions screen (real, revocable, backed by a sessions

table) +
disabled 2FA (ok to be "coming soon")
Support: Linking.openURL for all links
Logout: confirmation alert + clear tokens + invalidate session server-side +
navigate Welcome
Delete Account: double confirmation + POST /api/v1/accounts/delete-account
(soft-delete server-side – a `deletedAt` column, not a hard row delete,
unless the existing schema already does hard deletes elsewhere)

Also create if missing, and register in navigators:

ChangePasswordScreen.tsx → registered in settings stack
LoginSessionsScreen.tsx → registered in settings stack

7.7 Private Chat + Group Chat

Backend – Socket.IO (NOT Django Channels, NOT raw `ws`):

Socket.IO server attached to the same Express HTTP server in artifacts/api-server.

Namespaces/rooms (verify actual naming already used in the codebase, or establish

this convention if none exists):

Private chat: room per conversation, e.g. `conversation:
{conversationId}`

Group chat: room per group, e.g. `group:{groupId}`

Auth: a Socket.IO server-side middleware (`io.use(...)`) validates the JWT from

the handshake `auth.token` payload before allowing the connection. On invalid

token → reject the connection (`next(new Error('unauthorized'))`), which surfaces as a `connect\_error` on the client.

Authorization: before allowing a socket to `join` a conversation/group room,

verify server-side that the authenticated user is actually a participant/

member (via a Drizzle query against the conversation/group membership table).

Reject with a clear error event + disconnect if not authorized.

REST endpoints: standard chat CRUD (list conversations, message history, etc.)

in artifacts/api-server/src/routes/chats.ts and groups.ts, backed by Drizzle.

Frontend:

FriendChatScreen.tsx

socket.io-client as the primary channel, REST fallback for sending if needed

Typing indicators, read receipts, delivered status via Socket.IO events

KeyboardAvoidingView behavior="height" (Android; no-op on Web)

GroupChatScreen.tsx

Sender name above each incoming message

Real-time messages via Socket.IO room events

Member management, pin/unpin messages (role-gated to group admins, enforced

server-side)

CreateGroupStep1Screen.tsx + Step2

Searchable friends list → group details → CREATE GROUP

Also verify: friend request send/accept/reject/remove generates real notifications (DB rows, not just UI toasts).

7.8 Bidding Engine ← MOST CRITICAL FEATURE

Backend – check artifacts/api-server/src/routes/bids.ts :

POST /api/v1/bids	← buyer only
POST /api/v1/bids/:id/start (see below)	← buyer, schedules the auto-close
POST /api/v1/bids/:id/join only	← invited seller, pending status
POST /api/v1/bids/:id/reject	← seller
POST /api/v1/bids/:id/offer	← joined seller only
PATCH /api/v1/bids/:id/offer endsAt (server-enforced)	← joined seller, blocked after

POST /api/v1/bids/:id/select-winner ← buyer only, { offerId }
GET /api/v1/bids/:id ← returns live analytics

Server-side enforcement (non-negotiable, test by actually calling as the wrong role):

Rejected sellers → 403 on offer attempt (verify by actually trying it)

Offer edit after endsAt → 400

Buyer cannot bid on their own product → 400

Winner selection triggers the order creation flow (real DB transaction)

Auto-close mechanism (NO Celery/Redis available):

Use node-cron (or an existing equivalent scheduler already in the repo) running

a sweep every N seconds/minutes that finds bids where `endsAt <= now()` and

`status = 'active'`, closes them, and emits the relevant Socket.IO event to the

bid's room. This must survive server restarts (it's a DB-driven sweep, not an

in-memory setTimeout keyed off server uptime) – verify the sweep interval is

tight enough that bids close within an acceptable margin of `endsAt`.

Frontend:

CreateBidScreen.tsx – buyer only, product auto-fill, seller selection

SelectSellersScreen.tsx – channel list with checkboxes (all or specific sellers)

LiveBidScreen.tsx

Countdown timer (large Mono font, Accent color)

socket.io-client connected to the bid's room for real-time updates

(not polling, since Socket.IO already exists)

Analytics panel: offer count, lowest, highest, average – updated in real time

Offers list sorted by price

Timer hits 0 → auto-navigate to BuyerOfferSelectionScreen

BuyerOfferSelectionScreen.tsx ← CRITICAL FAILURE POINT (see Phase 1 item 2)

SellerBidRequestScreen.tsx – JOIN BID + REJECT BID buttons working

`SellerOfferScreen.tsx` – submit + update offer working

`SellerRejectBidScreen.tsx` – reject with reason working

7.9 Orders & Order Chat

Backend – check `artifacts/api-server/src/routes/orders.ts`:

`POST /api/v1/orders` ← buyer, after selecting winner

`PATCH /api/v1/orders/:id/accept` ← seller

`PATCH /api/v1/orders/:id/deliver` ← seller

`PATCH /api/v1/orders/:id/complete` ← buyer

`GET/POST /api/v1/orders/:id/chat`

Realtime: Socket.IO room per order, e.g. ``order:{orderId}``, for order chat
–

same auth-middleware + membership-check pattern as private/group chat.

On order accept → create the order conversation row + notify the buyer
(DB notification row + optional Socket.IO push).

Status machine (pending → accepted → in_progress → delivered → completed / cancelled)

must be enforced server-side; invalid transitions rejected with 400.

Frontend:

`OrderRequestScreen.tsx` – SEND ORDER REQUEST → `POST /api/v1/orders`

`OrderDetailScreen.tsx` – status-based buttons, WRITE REVIEW when completed

`OrderChatScreen.tsx` – ONLY ONE FILE, at `src/screens/orders/`

No duplicate at `src/screens/chat/OrderChatScreen.tsx`

Real-time messages via the order's Socket.IO room; private, two-participant channel

8. PHASE 3 – MEDIUM PRIORITY

1. **Loading skeletons** on every list/data screen (Home, Chat list, Channel list, Wishlist, Live Bid, Seller Analytics, Notifications, Search). Real shimmer/loading UI while `isLoading`, real error state with retry while `isError`, real empty state with a helpful message when data is empty.
2. **Error boundaries** wrapping the root navigator and each critical screen (bid, offer selection, order chat, review) so a JS error shows a recoverable fallback instead of a blank/white screen – this matters especially on the Web target.

3. **Standard API response shape** – confirm every Express endpoint returns a consistent success/error envelope (`{success, message, data, meta}` / `{success:false, message, errors, code}` or whatever consistent shape already exists in the codebase) and that the frontend error handling relies on it consistently (via the shared `lib/api-client-react` interceptor) rather than ad hoc per-screen parsing.

4. **Search + Profile + Notifications:**

SearchScreen.tsx

- Debounced search (300ms), real API results
- Recent searches + trending chips before typing
- Skeleton during search, empty state on no results

ProfileScreen.tsx

- Real data from GET `/api/v1/auth/me` + GET `/api/v1/profile/me/stats`
- Role-specific stat rows
- Edit profile image upload (via POST `/api/v1/uploads`, local disk)

NotificationsScreen.tsx

- Real notification list, mark read, mark-all-read, unread count, skeleton, empty state

5. **Navigation integrity:**

Open the app's MainNavigator (React Navigation v7) and verify:

- All screens are registered exactly once
- OrderChatScreen imports from 'screens/orders/OrderChatScreen'
- ReviewScreen is registered in the orders stack
- ChangePasswordScreen registered in settings stack
- LoginSessionsScreen registered in settings stack
- SellerAnalyticsScreen accessible from the More tab

Open the RootNavigator and verify:

- ErrorBoundary wraps the navigator
- SplashScreen is the first screen
- Correct conditional navigation (auth vs main)

6. **Socket.IO infrastructure:**

On the server (artifacts/api-server):

- A Socket.IO auth middleware (`io.use(...)`) validates the JWT from the handshake

- auth payload on every connection attempt

- Room-join handlers verify DB-level membership before allowing a join

Unauthorized connections/joins reject clearly (`connect_error` / a dedicated

`'unauthorized' event + disconnect`) – client-testable

On the client (`artifacts/o2o`), open the shared socket hook (e.g. `useSocket.ts`) and verify:

Connects with `{ auth: { token: accessToken } }`

Handles `'connect_error'` (e.g. invalid/expired token → refresh + reconnect)

Handles a server-emitted `'unauthorized'` event (→ navigate home)

Relies on `Socket.IO`'s built-in reconnection rather than a custom reconnect loop,

unless the existing codebase already has a deliberate custom pattern

Works on Android (native) and Web (browser) – NO iOS references

9. PHASE 4 – LOW PRIORITY (do last, only after everything above is verified working)

1. **Backend tests** – write real tests (Jest + Supertest, or whatever test runner is already configured for `artifacts/api-server`) for auth, permissions (role boundaries – buyer vs seller vs owner), the full bidding flow, order status transitions, and review validation. Target $\geq 80\%$ coverage on core routes. Every test must actually assert real behavior, not just call an endpoint and check for 200.
 2. **Frontend tests** – unit tests (Jest + React Native Testing Library) for shared atoms (button, star rating, input) and the auth context (token storage/refresh/logout). If time allows, add Playwright E2E for the Web target covering: register → login → create bid → submit offer → select winner → review. **Do not** write or reference any iOS test configuration – Android + Web only, and only if there's runway after everything above is solid.
 3. **Monitoring** – if Sentry (or an equivalent already partially wired in the codebase) exists, finish wiring it: backend captures 500s, frontend captures crashes on Android and Web (not iOS), user context is set on login and cleared on logout.
-

10. FULL FEATURE CHECKLIST – VERIFY EVERY ROW, ANDROID + WEB ONLY

Go through this list top to bottom as a final verification pass. For each row, actually operate the feature (don't just read code) on both Android (emulator/device via React Native CLI) and Web (Vite dev/build)

before marking it done.

Auth & Account

- ☐ Splash → Welcome → Login → Signup → Forgot Password (email → OTP → reset) all functional
- ☐ Register/login validation enforced server-side (unique username/email/mobile, password rules, role must be buyer/seller only – never self-assignable admin)
- ☐ JWT issued, stored, refreshed, invalidated on logout – Android ✓ Web ✓
- ☐ Edit profile, change password, change email/mobile (or explicit "coming soon" only if spec allows it)
- ☐ Login sessions list + per-session revoke works against real DB session records

Social

- ☐ Friend request send/accept/reject/remove, with real notifications generated
- ☐ Private chat: real-time messages, typing indicator, read/delivered status, all via Socket.IO
- ☐ Group chat: create (2-step flow), send messages, member management, pin/unpin messages (role-gated to group admins)

Channels & Products

- ☐ Seller-only: create channel, post product (images + custom key/value details), edit/delete own product
- ☐ Buyer: browse channels/products, cannot post, can follow/unfollow
- ☐ Repost: real DB duplication + editable prefilled form + republish persists changes
- ☐ Product detail screen: real view counter increment, real wishlist toggle

Wishlist & Bidding

- ☐ Add/remove wishlist items call real endpoints and update counts server-side
- ☐ Create bid → select sellers (all or specific) → live 30-min timer → real-time analytics (offer count, lowest/highest/avg price) via Socket.IO → seller join/reject → seller offer submit/edit (time-boxed server-side) → buyer selects winner → order created
- ☐ Rejected sellers verified (by actually testing) unable to submit offers
- ☐ Bid auto-closes at timer expiry via the node-cron (or equivalent) sweep job, not a frontend-only timer

Orders & Reviews

- ☐ Order status transitions enforced server-side, invalid transitions rejected

- ☐ Order chat private and functional (Socket.IO room, membership-checked)
- ☐ Review: stars + text + up to 3 photos, blocked until order truly completed, one review per order, seller gets a real notification

Settings & Notifications

- ☐ Every settings toggle persists to a real DB row (via Drizzle) and reflects on reload
- ☐ Notifications list, unread count, mark-read/mark-all-read all work against real data
- ☐ Theme switch (light/dark/system) persists locally (AsyncStorage/localStorage) and applies across the app

Analytics

- ☐ Seller analytics stats and charts are 100% real-data-driven (Drizzle queries), verified against manual DB queries

Cross-Cutting Quality

- ☐ Loading, error, and empty states present on every list/data screen
- ☐ No console errors/warnings in Android logs or the Web browser console during normal use
- ☐ No mock data files are imported anywhere in production code paths (mock files may remain in a `/mock` folder only if completely unused by the shipped app — otherwise delete them)
- ☐ No stray Django/Python/AWS/Redis/Expo references anywhere in the monorepo
- ☐ Responsive layout: mobile width on Android, and correctly adapts to tablet/desktop breakpoints in the Web build (sidebar/grid layout at wider widths)



11. DESIGN SYSTEM ENFORCEMENT

While fixing functionality, also verify (and correct if drifted) that every screen uses the **same** design tokens — do not introduce new one-off colors, spacing, or fonts:

Fix any screen with hardcoded hex colors → use design system constants

Fix any screen with arbitrary spacing → use 4/8/12/16/20/24/32/40/48/64 scale

Fix any screen with raw `fontSize` → use typography constants

Fix all buttons → height 52px, borderRadius 16, correct variant colors

Fix all inputs → height 52px, borderRadius 12, label above field

Fix all cards → borderRadius 16, shadow, 16px padding

Design tokens:

Primary: '#0057FF', Accent: '#FF6B00', Success: '#00B37D'
Danger: '#E53935', Surface: '#FFFFFF', Background: '#F4F6FA'
Border: '#E2E8F0', TextPrimary: '#0D1117', TextSecondary: '#6B7280'

Also verify:

- Colors, typography scale, spacing scale, border-radius scale, and shadows all come from one shared theme/constants module – no screen defines its own hardcoded hex colors or magic-number spacing.
 - Buttons, inputs, cards, avatars, badges, chips all use the shared atom components – no screen reimplements its own button/card style inline.
 - Icon usage is consistent in size and color source across screens.
 - The admin panel(`artifacts/admin-panel`) should visually align with the same token philosophy where practical, even though it's a separate Vite/React app – don't let it drift into an unrelated arbitrary style if it's meant to share branding with the mobile/web app.
-



12. ANDROID-SPECIFIC FIXES

Every form screen: `KeyboardAvoidingView` `behavior="height"`

Image picker: request the appropriate Android storage/media permission before

opening (via `PermissionsAndroid` or the bare-RN permissions library already in

the repo – NOT `expo-permissions`)

→ show rationale if denied

→ fallback message if permanently denied

`FlatList`: `keyExtractor` always returns unique stable string IDs

Hardware back button: does not break navigation (no dead ends)

Status bar: React Native's built-in `StatusBar` component with correct colors

(NOT `expo-status-bar`)

Touch targets: minimum 44dp height for all interactive elements

FCM: whatever bare-RN Firebase Android messaging integration already exists in

the repo (e.g. `@react-native-firebase/messaging`) is correctly configured (`google-services.json` present, Android manifest entries correct)



13. WEB-SPECIFIC FIXES

Desktop (>1024px): sidebar nav (240px) replaces bottom tabs

Sidebar items: Home, Chat, Groups, Channels, Profile, Settings

Cards: 2-col grid on tablet, 3-col on desktop

Chat: split view on desktop (list left 35%, chat right 65%)

File upload: the bare-RN image picker library falls back to a plain

`<input type="file">` on the Web platform branch (Platform.OS === 'web')

Token storage: localStorage (not AsyncStorage) on Web

Realtime: socket.io-client works over the browser transport automatically;

verify the handshake auth payload still carries the JWT correctly on Web

Linking.openURL: opens in a new tab on Web

Hover states: all pressable elements have hover feedback

Focus outlines: keyboard navigation works

No horizontal scroll on any screen at any viewport width

Browser back/forward: works correctly with React Navigation's Web support

Vite build: confirm React Native Web aliasing/resolution is correctly configured

in vite.config.ts so RN components compile for the browser target

14. BACKEND CORRECTNESS CHECK & FINAL COMMANDS

For `artifacts/api-server` (Express + TypeScript + Drizzle/Postgres):

- All Drizzle schema tables use proper unique constraints where required (username, email, mobile, one-review-per-order, one-wishlist-item-per-buyer-per-product, etc.)
- All list endpoints are paginated – no endpoint returns an unbounded query
- All file uploads (Multer) are validated server-side by actual MIME type (not just file extension) and size-limited before being written to local disk
- All role-gated endpoints return 403 to the wrong role – verify this by literally calling them as the wrong role and confirming the rejection
- The Socket.IO auth middleware rejects unauthenticated/unauthorized connections with a clear error
- The node-cron (or equivalent) bid auto-close sweep, OTP cleanup, and any other required scheduled job are actually registered and actually running, not just defined and never scheduled

Run from the repo root / `artifacts/api-server` :

```
# TypeScript must compile clean
```

```
pnpm --filter api-server run typecheck # tsc --noEmit, or equivalent sc
```

```
# Must: 0 errors
```

```

# ESBUILD production build must succeed
pnpm --filter api-server run build
# Must: 0 errors

# Drizzle migrations must be up to date and apply cleanly to a fresh DB
pnpm --filter db run drizzle:generate      # or the actual configured script
pnpm --filter db run drizzle:migrate
# Must: no pending/unapplied migrations, no schema drift

# Verify JWT config
node -e "
require('dotenv').config();
console.log('Access secret set:', !!process.env.JWT_ACCESS_SECRET);
console.log('Refresh secret set:', !!process.env.JWT_REFRESH_SECRET);
console.log('Access expiry:', process.env.JWT_ACCESS_EXPIRY);
console.log('Refresh expiry:', process.env.JWT_REFRESH_EXPIRY);
"

# Verify Socket.IO namespaces/handlers register without error
pnpm --filter api-server run start &
sleep 3
curl -s http://localhost:PORT/health      # or whatever health endpoint exists
kill %1

# Verify scheduled jobs are registered (check the cron registration file)
grep -rn "cron.schedule" artifacts/api-server/src

```

Fix anything that fails.



15. MOBILE / WEB BUILD CHECKS

```

# Verify no iOS references remain anywhere in the monorepo
test -d artifacts/o2o/ios && echo "FAIL: ios/ still exists"
grep -rn "Platform.OS === 'ios'" artifacts/o2o/src
grep -n "\"ios\"" artifacts/o2o/package.json
grep -ri "expo" artifacts/o2o/package.json artifacts/o2o/src      # should not find anything

# Android build must succeed (adjust to the repo's actual script names)
pnpm --filter o2o run android:build      # e.g. `cd android && ./gradlew assemble`
# Must complete with 0 errors

```

```
# Web build via Vite must succeed
pnpm --filter o2o run build:web
# Must complete with 0 errors, 0 warnings

# Admin panel build via Vite must succeed
pnpm --filter admin-panel run build
# Must complete with 0 errors, 0 warnings

# Full monorepo type-check
pnpm -r run typecheck
# Must: 0 errors across all packages
```



16. FINAL PACKAGING REQUIREMENTS

Once every item above is verified (not assumed) working:

1. Remove all dead code: unused files, commented-out blocks of old implementations, unused imports, leftover mock data files that are no longer referenced.
2. Remove all iOS artifacts per Section 5, and all Django/Python/AWS/Redis/Expo artifacts per Section 1 – do a final grep pass to confirm zero remaining references.
3. Make sure `.env.example` files (for `artifacts/api-server` and `artifacts/o2o` if it needs any build-time env vars) list every required environment variable with empty/placeholder values – no real secrets committed.
4. Confirm the API server builds and starts cleanly, Drizzle migrations apply cleanly to a fresh PostgreSQL database, the mobile app builds/runs cleanly on Android, the Web target builds cleanly via Vite, and the admin panel builds cleanly via Vite.
5. Document exactly how to run the project from a clean clone on Replit in a `SETUP.md`: install steps (`pnpm install` at root), env vars, DB migration commands, and run commands for the API server, the mobile app (Android + Web), and the admin panel, in that order.
6. Package the entire corrected monorepo (excluding `node_modules`, any `dist` / `build` output, and local upload directories – include a note in `SETUP.md` on how to regenerate them) as a single downloadable archive, or leave it as a clean, immediately-runnable Replit workspace if that's the preferred delivery format.
7. Write a final `CHANGELOG_AUDIT_FIXES.md` summarizing, per the priority tiers above, exactly what was found broken/missing/off-stack and exactly what was done to fix it – this is your evidence that the audit was real and not skipped.

Structure (monorepo, matches the confirmed stack – do not restructure into a Django-style layout):

```

o2o/
├─ lib/
│   └─ db/                                ← Drizzle schema, migrations, drizzle-kit
config
├─ └─ api-client-react/                  ← shared React Query hooks / API client
├─ artifacts/
│   └─ api-server/
│       └─ src/
│           └─ routes/                  ← auth, channels, products, bids, orders,
│               │                               reviews, wishlist, analytics,
│               │                               accounts, uploads
│               └─ sockets/              ← Socket.IO auth middleware + event
handlers
│   └─ └─ jobs/                          ← node-cron scheduled jobs (bid auto-
close, etc.)
│       └─ └─ middleware/                ← auth, error handling, rate limiting
│           └─ └─ index.ts               ← Express + Socket.IO server bootstrap
│               └─ uploads/              ← local disk storage for uploaded files
(served statically)
│   └─ └─ package.json
│       └─ └─ .env.example
│           └─ o2o/
│               └─ src/
│                   └─ api/              ← screen-facing API hooks (built on
lib/api-client-react)
│   └─ └─ └─ components/                 ← atoms, molecules, organisms
│       └─ └─ └─ context/                ← AuthContext, ThemeContext, etc.
│           └─ └─ └─ hooks/              ← useSocket, etc.
│               └─ └─ └─ navigation/      ← RootNavigator, MainNavigator (React
Navigation v7)
│   └─ └─ └─ └─ screens/                 ← all screen files
│       └─ └─ └─ android/                ← Android native project (RN CLI bare)
│           └─ └─ └─ (no ios/ folder)
│               └─ App.tsx
│                   └─ package.json      ← no "ios" script
│                       └─ vite.config.ts ← Web target bundling
├─ └─ admin-panel/
│   └─ └─ src/
│       └─ └─ package.json
│           └─ vite.config.ts
└─ pnpm-workspace.yaml

```

- └─ package.json
- └─ AUDIT_REPORT.md
- └─ CHANGELOG_AUDIT_FIXES.md
- └─ SETUP.md
- └─ README.md

Deliver the final package plus `AUDIT_REPORT.md`, `CHANGELOG_AUDIT_FIXES.md`, `SETUP.md`, and `README.md` as the output of this task.

17. README.md CONTENT (include in final package)

```
# O2O – Production Ready Application
## Platforms: Android + Web

### Prerequisites
- Node.js (LTS)
- pnpm
- PostgreSQL (local or hosted instance you control – no managed AWS servi

### Root Setup
1. pnpm install                                (installs all workspace packages)
2. cp artifacts/api-server/.env.example artifacts/api-server/.env → fill

### Database
1. pnpm --filter db run drizzle:generate
2. pnpm --filter db run drizzle:migrate

### API Server
1. pnpm --filter api-server run dev           → starts Express + Socket.IO serv
2. Server runs at: http://localhost:<PORT>
3. Uploaded files served from: http://localhost:<PORT>/uploads

### Mobile App – Android
1. pnpm --filter o2o run android              → builds & runs on emulator/devic

### Mobile App – Web
1. pnpm --filter o2o run dev:web              → development (Vite)
2. pnpm --filter o2o run build:web            → production build (Vite)

### Admin Panel
1. pnpm --filter admin-panel run dev          → development (Vite)
```

```
2. pnpm --filter admin-panel run build → production build (Vite)
```

```
### Required Environment Variables (artifacts/api-server/.env)
```

```
DATABASE_URL=postgres://user:password@host:5432/o2o_db
```

```
JWT_ACCESS_SECRET=
```

```
JWT_REFRESH_SECRET=
```

```
JWT_ACCESS_EXPIRY=15m
```

```
JWT_REFRESH_EXPIRY=30d
```

```
PORT=8000
```

```
CORS_ORIGIN=http://localhost:8081
```

```
UPLOAD_DIR=./uploads
```

```
MAX_UPLOAD_SIZE_MB=10
```

```
FIREBASE_SERVICE_ACCOUNT_JSON= (Android FCM push)
```

```
EMAIL_HOST=smtp.gmail.com
```

```
EMAIL_PORT=587
```

```
EMAIL_HOST_USER=
```

```
EMAIL_HOST_PASSWORD=
```

```
EMAIL_USE_TLS=true
```

```
DEFAULT_FROM_EMAIL=noreply@o2o.app
```

```
FRONTEND_URL=http://localhost:8081
```

```
### Tech Stack
```

- Monorepo: pnpm workspaces, TypeScript throughout
- Backend: Node.js + Express v5 + TypeScript
- Database: PostgreSQL + Drizzle ORM (+ drizzle-zod for validation)
- File storage: local disk on the API server, served via Express static r
- Realtime: Socket.IO
- Scheduled jobs: node-cron
- Auth: JWT (jsonwebtoken), access + refresh
- Mobile/Web frontend: React Native CLI (bare) — Android + React Native W
- Admin panel: React 19 + Vite + Wouter + Recharts
- Push: Firebase Cloud Messaging (Android only)

18. SUCCESS CRITERIA

The prompt is complete when ALL of the following are true:

ANDROID:

- ✓ App builds and launches without errors via React Native CLI
- ✓ JWT auth: login, register, logout, refresh all work
- ✓ No iOS code, no ios/ folder, anywhere in the project
- ✓ No Expo packages/config anywhere in the project

- ✓ All screens load with real API data (zero mock data)
- ✓ Full bidding flow: create bid → live timer → offers → select winner → order
- ✓ Private chat + group chat real-time via Socket.IO
- ✓ Wishlist add and remove both work with API calls
- ✓ ReviewScreen fully implemented (not placeholder)
- ✓ SellerAnalyticsScreen shows real numbers from backend
- ✓ SettingsScreen all toggles call real APIs
- ✓ selectWinnerMutation is defined and wired in BuyerOfferSelectionScreen
- ✓ Only one OrderChatScreen file (in orders/)
- ✓ Loading skeletons on all list screens
- ✓ Error states with retry on all screens
- ✓ Hardware back button works correctly everywhere

WEB:

- ✓ Vite web build completes with 0 errors
- ✓ All Android features work in browser
- ✓ Sidebar navigation visible at desktop width
- ✓ JWT stored in localStorage
- ✓ Socket.IO connects in browser
- ✓ File uploads work via browser file picker
- ✓ No horizontal scroll at any viewport width

BACKEND:

- ✓ TypeScript compiles with 0 errors, ESBUILD build succeeds
- ✓ All Drizzle migrations applied, no schema drift
- ✓ All API endpoints respond correctly
- ✓ Role boundaries enforced server-side (verified by testing as wrong role)
- ✓ JWT access/refresh flow working correctly
- ✓ Socket.IO auth middleware rejects invalid/unauthorized connections
- ✓ node-cron sweep closes bids at timer expiry automatically
- ✓ Analytics computed from real Drizzle/DB queries
- ✓ File uploads written to local disk and served correctly (no S3 dependency)

STACK INTEGRITY:

- ✓ No Django, Python, AWS/S3, Redis, Celery, or Expo code/config remains anywhere
- ✓ No new major dependency was added without first checking for an existing equivalent

ADMIN PANEL:

- ✓ Builds cleanly via Vite
- ✓ Uses the same JWT-authenticated API, no mock data

PACKAGING:

- ✓ Final package produced with the structure in Section 16
 - ✓ `AUDIT_REPORT.md`, `CHANGELOG_AUDIT_FIXES.md`, `SETUP.md`, `README.md` all included
-

19. THINGS TO NEVER DO WHILE EXECUTING THIS PROMPT

- Never say a feature is "done" without having actually exercised it (tapped the button, sent the request, checked the DB row).
 - Never leave a `TODO`, `FIXME`, `console.log("test")`, or commented-out block in the final delivered code.
 - Never generate placeholder code, `TODO` comments, or stub functions.
 - Never write, reference, or leave behind any iOS-specific code, config, or documentation.
 - Never introduce Django, Python, AWS/S3, Redis, Celery, or Expo — anywhere, for any reason, even "temporarily."
 - Never add a new major dependency (ORM, realtime library, cloud SDK, charting library, etc.) without first confirming nothing equivalent already exists in the relevant `package.json`.
 - Never invent a feature that isn't in the source documents/spec just to "add value" — stay scoped to completing what's specified.
 - Never silently skip a checklist item because it seems hard — if something is genuinely out of scope for this pass, say so explicitly in `AUDIT_REPORT.md` rather than omitting it.
-

End of prompt. Work through: Phase 0 (Intake) → iOS Removal → Phase 1 (Critical) → Phase 2 (High Priority) → Phase 3 (Medium) → Phase 4 (Low) → Full Feature Checklist verification → Design System check → Android/Web-specific fixes → Backend correctness check → Mobile/Web build checks → Final Packaging, in that order, without skipping ahead.

020 Master Prompt — Stack: Node/Express/TypeScript + Drizzle/PostgreSQL + React Native CLI + Socket.IO (pnpm monorepo)

Platforms: Android ✓ Web ✓ iOS ✗ (completely removed)

Explicitly excluded: Django ✗ Python ✗ AWS/S3 ✗ Redis ✗ Celery ✗ Expo ✗

Output: production-ready monorepo package + `AUDIT_REPORT.md` + `CHANGELOG_AUDIT_FIXES.md` + `SETUP.md` + `README.md`

